

NEURAL NETWORKS, PERCEPTRON & DEEP LEARNING

Day 12 Task– Neural Networks – 27/06/2026

Name: Manish Giri | Reg. No.: 137256 | Email: manishgiri8101@gmail.com | [[LinkedIn](#)]

PART A — Concept Review

Q1. Define Perceptron in your own words.

Answer:

A Perceptron is the simplest unit of a neural network — a mathematical model inspired by a biological neuron. It takes multiple inputs, multiplies each by a weight (importance value), sums them up, and passes the result through an activation function to produce an output (typically 0 or 1).

Example:

Imagine you're deciding whether to carry an umbrella. Your inputs might be: Is it cloudy? (yes=1) + Is it raining? (yes=1) + Did the forecast say rain? (yes=1). Each factor has a weight based on importance. If the weighted sum exceeds a threshold, the perceptron outputs 1 (carry umbrella). That's exactly how a perceptron works!

Q2. List three key differences between a Perceptron and a Deep Neural Network.

Answer:

- Structure: A Perceptron has only ONE layer (input → output), while a Deep Neural Network (DNN) has MULTIPLE hidden layers between input and output.
- Complexity: A Perceptron can only solve linearly separable problems (e.g., AND/OR gates), whereas a DNN can model complex, non-linear patterns (e.g., recognizing faces in photos).
- Learning Power: A Perceptron uses a simple update rule, but a DNN uses backpropagation across many layers, allowing it to learn intricate representations of data.

Example:

Perceptron: Can decide if an email is spam based on one keyword. DNN: Can analyze the full email, tone, sender history, and links together — like Gmail's spam filter.

Q3. Explain what Gradient Descent does in a neural network.

Answer:

Gradient Descent is the optimization algorithm that helps a neural network learn by minimizing the error (loss). It calculates how much the error changes with respect to each weight (the gradient), then adjusts

the weights slightly in the direction that reduces error. This is repeated many times until the model reaches minimum loss.

Example:

Think of rolling a ball down a hilly landscape blindfolded. The ball (your model) tries to reach the lowest valley (minimum error). At each step, it feels which direction is downhill (gradient) and takes a small step that way (learning rate). Gradient Descent is that guiding process.

Q4. Why is backpropagation essential for learning?

Answer:

Backpropagation (Backward Propagation of Errors) is the mechanism that distributes the blame for prediction errors back through the network. After a forward pass produces a wrong answer, backpropagation calculates how much each weight in every layer contributed to that error (using the chain rule of calculus), allowing all weights to be updated accordingly.

Example:

Without backpropagation, a deep network with 10 hidden layers would have no way of knowing whether an error was caused by Layer 2 or Layer 8. Backprop makes this attribution possible — like tracing a miscommunication in a telephone game back to whoever started it.

PART B — Titanic Survival Case Study

Q1. Identify five features used to predict survival in the Titanic dataset.

Answer:

- Pclass (Passenger Class) — 1st, 2nd, or 3rd class ticket, indicating socioeconomic status.
- Sex — Male or Female, a critical factor (women were given priority in lifeboats).
- Age — Children were often prioritized; age affected survival chances significantly.
- SibSp / Parch (Family Members Aboard) — Traveling with family affected decisions and survival.
- Fare — The price paid for the ticket, indirectly indicating class and deck location.

Example:

A 28-year-old female in 1st class who paid a high fare and traveled alone would be predicted high survival probability. A 40-year-old male in 3rd class with a low fare would receive a low survival probability — the model learned these patterns from historical data.

Q2. Describe how Gradient Descent helps the model improve its predictions.

Answer:

In the Titanic model, Gradient Descent starts with random weights and makes initial (often wrong) survival predictions. It calculates the loss (e.g., binary cross-entropy) between predictions and actual survival

outcomes. It then computes the gradient — how much each weight influenced the wrong prediction — and nudges weights to reduce that error. Over thousands of iterations (epochs), the model steadily improves its accuracy.

Example:

Iteration 1: Model predicts 'survived' for everyone → High loss. Gradient Descent adjusts weights, placing more importance on 'sex=female' and 'pclass=1'. Iteration 100: Model now correctly predicts that most 3rd-class males didn't survive. Loss drops significantly.

Q3. What patterns might the network learn about survival?

Answer:

- Women had significantly higher survival rates than men ('women and children first' policy).
- Passengers in 1st class had higher survival rates due to better access to lifeboats.
- Children (age < 15) had higher survival rates than adult men.
- Passengers traveling alone had lower survival rates than those with families.
- Higher ticket fares correlated with higher survival (linked to cabin deck location).

Example:

The network might learn a rule like: IF sex=female AND pclass=1 THEN survival probability ≈ 97%. IF sex=male AND pclass=3 AND age>18 THEN survival probability ≈ 13%. These aren't hard-coded rules — the network discovers them automatically from data.

Q4. Sketch a simple diagram showing how data flows through layers.

Answer:

A forward-pass data flow through a neural network for Titanic prediction:

INPUT LAYER → HIDDEN LAYER 1 → HIDDEN LAYER 2 → OUTPUT LAYER

- Input Layer: Takes raw features — [Pclass=1, Sex=female, Age=28, Fare=72.5, Family=0]
- Hidden Layer 1 (e.g., 64 neurons): Detects low-level patterns — relationships between age & sex.
- Hidden Layer 2 (e.g., 32 neurons): Combines patterns — class + fare + gender interactions.
- Output Layer (1 neuron, sigmoid): Outputs survival probability → 0.89 (89% chance survived).

Example:

Each arrow between layers represents weighted connections. Neurons apply: $\text{Output} = \text{Activation}(\sum(\text{weight} \times \text{input}) + \text{bias})$. The sigmoid function at the final layer squeezes output between 0 and 1, giving a probability.

PART C — Real-World Applications (Matching)

Match each domain with its neural-network use case:

Domain	Application	Brief Explanation / Example
Healthcare	Tumor Detection	CNNs analyze MRI/CT scans to identify malignant tumors with accuracy rivaling radiologists. Example: Google's DeepMind AI detects breast cancer from mammograms with >99% accuracy.
Finance	Market Trend Prediction	LSTMs (Long Short-Term Memory networks) analyze historical price data, news sentiment, and volume to predict stock market movements. Example: Hedge funds use RNNs to trade algorithmically.
Space Science	Satellite Image Analysis	CNNs process thousands of satellite images to detect deforestation, map terrain, track weather patterns, and identify astronomical objects. Example: NASA uses neural nets to classify galaxies from telescope data.
Education	Adaptive Learning Systems	Neural networks analyze a student's performance patterns, quiz results, and time-on-task to recommend personalized learning paths. Example: Duolingo's AI adapts lesson difficulty in real-time based on user errors.

PART D — Think & Reflect

"From simple neurons to deep insights — AI is teaching machines to think."

Write a short paragraph (5–6 lines) on what this statement means to you.

Answer:

This statement beautifully captures the essence of the journey from simple mathematical units to sophisticated intelligence. A single perceptron — modeled after a biological neuron — can only make a basic yes/no decision. Yet, when thousands or millions of such neurons are connected in layers and trained on real data using techniques like backpropagation and gradient descent, something remarkable emerges: the ability to recognize faces, translate languages, diagnose diseases, and predict survival on the Titanic. The phrase 'AI is teaching machines to think' is profound because the machine doesn't think

the way humans do — instead, it learns statistical patterns so complex and nuanced that the outputs resemble intelligent thought. It reminds me that intelligence, whether biological or artificial, may ultimately be about connections, weights, and learning from experience. What begins as a simple threshold function evolves, layer by layer, into systems that perceive the world in ways we are only beginning to understand.

Example:

Just as the human brain starts with simple neurons firing and over years of learning becomes capable of writing poetry or solving equations, a deep neural network starts with random weights and through training on data, emerges with capabilities its designers never explicitly programmed — that is the power and wonder of deep learning.